SQL Code

```sql
-- SQL CLAUSES
-- WHERE,ORDER BY ,GROUP BY, HAVING
-- WHERE -USED TO FILTER THE RECORDS BASED ON CONDITION
SELECT*FROM SALES;
SELECT*FROM employee.sales;
select*from employee.sales WHERE REGION ="SOUTH";
SELECT*FROM employee.sales WHERE SALES>1000;
-- TASK
-- EXTRACT THE RECORDS WHERE QUANTITY IS 5 AND LESSER THAN 5
SELECT*FROM employee.sales WHERE QUANTITY<=5;
-- OR
-- EXTRACT THE RECORDS EXCEPT REGION CENTRAL
SELECT*FROM employee.sales WHERE REGION="EAST" OR REGION="SOUTH";
-- AND
-- EXTRACT THE RECORDS RELATED TO CATEGORY FURNITURE WHERE QUANTITY>5
SELECT*FROM employee.sales WHERE CATEGORY= "FURNITURE" AND QUANTITY>5;
-- TASK
-- EXTRACT THE RECORDS RELATED TO FURNITURE AND TECHNOLOGY WHERE SALES>500
SELECT*FROM employee.sales WHERE CATEGORY ="FURNITURE" OR CATEGORY="TECHNOLOGY" AND
SALES>500;
--- DISTINCT--- UNIQUE VALUE
SELECT DISTINCT (REGION) FROM employee.sales;
SELECT DISTINCT (QUANTITY)FROM employee.sales;
-- NOT
SELECT*FROM employee.sales WHERE REGION ="SOUTH";
SELECT*FROM employee.sales WHERE NOT REGION="SOUTH";
-- ALIASES(TEMPORARY NAME)
-- KEYWORD-- AS
SELECT REGION AS AREA FROM employee.sales;
SELECT* FROM employee.sales;
-- ORDER BY
-- ASCENDING ORDER(ASC)
-- DESCENDING ORDER(DESC)
SELECT *FROM employee.sales ORDER BY ORDERDATE ASC;
SELECT*FROM employee.sales ORDER BY ORDERDATE DESC;
SELECT*FROM employee.sales ORDER BY QUANTITY;
-- TASK
-- EXTRACT THE RECORDS IN DESC ORDER FOR THE QUANTITY>5
SELECT*FROM employee.sales WHERE QUANTITY>5 ORDER BY QUANTITY DESC;
-- IS NULL / IS NOT NULL
SELECT *FROM employee.sales WHERE DISCOUNT IS NULL;
SELECT*FROM employee.sales WHERE QUANTITY IS NOT NULL;
-- AGGREGATE FUNCTION
-- SUM
-- EXTRACT OVERALL SUM OF SALES
SELECT SUM(SALES)FROM employee.sales;
-- EXTRACT SUM OF employee.sales FOR CATEGORY TECHNOLOGY
SELECT SUM(SALES) FROM employee.sales WHERE CATEGORY ="TECHNOLOGY";
-- EXTRACT SUM OF SALES AND SUM OF QUANTITY FOR REGION ENDING WITH table
SELECT SUM(SALES),SUM(QUANTITY)FROM employee.sales WHERE REGION  LIKE "%T";
```

```sql
-- AVEREAGE
-- AVG QUANTITY
SELECT AVG (QUANTITY)FROM employee.sales;
-- COUNT-- COUNT THE NUMBER OF AVAILABLE RECORDS
SELECT COUNT(*) FROM employee.sales;
-- GROUP BY
-- ON ONE COLUMN
SELECT MIN(QUANTITY),MAX(QUANTITY),REGION FROM employee.sales GROUP BY REGION ORDER BY MAX(QUANTITY);
-- ONLY THE RECORDS WHERE AVG (SALES)>400
SELECT AVG (SALES),STATE FROM employee.sales GROUP BY STATE;
-- HAVING
SELECT AVG (SALES),STATE FROM employee.sales GROUP BY STATE HAVING AVG(SALES)>400;
SELECT AVG (SALES) ,REGION FROM employee.sales WHERE SALES>500 GROUP BY REGION HAVING AVG (SALES)>1000
ORDER BY AVG(SALES)DESC LIMIT 1;
--- SUBQUERY
CREATE TABLE CUSTOMER_S1 (C_ID INT,C_NAME VARCHAR(10));
INSERT INTO CUSTOMER_S1 VALUES(1,"JOHN"),(2,"ROBERT"),(3,"DAVID"),(4,"DAVID"),(4,"JOHN"),(5,"BETTY");
CREATE TABLE ORDERS_11(O_ID INT,AMT INT,C_ID INT);
INSERT INTO ORDERS_11 VALUES(1,1000,4),(2,3900,10),(3,2000,3),(4,2000,1),(5,100,2) ;
USE EMPLOYEE;
-- JOINS
CREATE TABLE EMPLOYEE_21(E_ID INT,E_NAME VARCHAR(10),SALARY INT,D_ID VARCHAR(5));
INSERT INTO EMPLOYEE_21 VALUES(11,"JOHN",25000,"D1"),(12,"ASH",34000,"D2"),(13,"KRISH",20000,"D3"),(14,"RAM",45000,"D4"),(15,"BOB",50000,"D5");
CREATE TABLE DEPARTMENT_31 (D_ID VARCHAR(5),D_NAME VARCHAR(15));
INSERT INTO DEPARTMENT_31 VALUES("D1","HR"),("D2","SALES"),("D3","MANAGER"),("D4","IT"),("D5","FINANCE");
-- INNER JOIN
SELECT*FROM EMPLOYEE_21 INNER JOIN DEPARTMENT_31 ON EMPLOYEE_21.D_ID=DEPARTMENT_31.D_ID ;
-- WITH ALIASES
-- EMP TO EMPLOYEE_21
-- DEPT TO DEPARTMENT_31
SELECT*FROM EMPLOYEE_21 AS EMP INNER JOIN DEPARTMENT_31 AS DEPT ON EMP. D_ID =DEPT.D_ID;
-- INNER JOIN AND GET E_NAME,D_ID  FROM EMPLOYEE_21 TABLE
-- D_NAME FROM DEPARTMENT_31 TABLE
SELECT EMPLOYEE_21.E_NAME,EMPLOYEE_21.D_ID,DEPARTMENT_31.D_NAME FROM EMPLOYEE_21 INNER JOIN DEPARTMENT_31 ON EMPLOYEE_21.D_ID= DEPARTMENT_31.D_ID;
-- RIGHT JOIN
SELECT*FROM EMPLOYEE_21 RIGHT JOIN DEPARTMENT_31 ON EMPLOYEE_21.D_ID= DEPARTMENT_31.D_ID;
-- LEFT JOIN
SELECT *FROM EMPLOYEE_21 LEFT JOIN DEPARTMENT_31 ON EMPLOYEE_21.D_ID=DEPARTMENT_31.D_ID UNION SELECT*FROM EMPLOYEE_21 RIGHT JOIN DEPARTMENT_31 ON EMPLOYEE_21.D_ID= DEPARTMENT_31.D_ID;
SELECT*FROM DEPARTMENT_31;
SELECT* FROM DEPARTMENT_31 AS D1 JOIN DEPARTMENT_31 AS D2 ON D1.D_ID= D2.D_ID;
-- CROSS JOIN
SELECT*FROM EMPLOYEE_21;
SELECT*FROM DEPARTMENT_31;
```

```sql
SELECT* FROM EMPLOYEE_21 CROSS JOIN DEPARTMENT_31;
-- INTERVIEW Questions
-- 1) What is the difference between WHERE and HAVING ?
-- WHERE and HAVING are SQL clauses used to filter data.
-- WHERE: Filters individual records before grouping.
-- HAVING- Filters grouped records after applying the GROUP BY clause.
-- The main difference is that WHERE is used to filter rows, whereas HAVING is used to filter groups.
-- 2) What are the different types of joins in SQL?
-- Joins are used to combine records from two or more tables based on a related column.
-- The different types of joins are:
-- 1. INNER JOIN: Returns only matching records from both tables.
-- 2. LEFT JOIN: Returns all records from the left table and matching records from the right table.
-- 3. RIGHT JOIN: Returns all records from the right table and matching records from the left table.
-- 4. FULL OUTER JOIN: Returns all records from both tables, including matching and non-matching records.
-- 5. CROSS JOIN: Returns all possible combinations of records from two tables.
-- 6. SELF JOIN: Joins a table with itself to compare records within the same table.
-- 3). How do you calculate Average Revenue Per User (ARPU) in SQL?
-- ARPU stands for Average Revenue Per User. It is a business metric used to calculate the average revenue generated by each individual user during a specific period.

-- Formula:

-- ARPU = Total Revenue / Total Number of Unique Users
-- It helps businesses understand how much revenue they generate from each customer on average.

-- ARPU is commonly used in e-commerce, telecommunications, subscription-based businesses, and digital platforms.
-- 4). What are subqueries in SQL?
-- A subquery is a query written inside another SQL query. It is also known as a nested query or inner query.

-- The result of the inner query is used by the outer query to retrieve the required information.

-- Types of subqueries:
-- 1. Single-row subquery
-- 2. Multiple-row subquery
-- 3. Correlated subquery
-- 4. Nested subquery
-- Subqueries are useful when we need to perform complex data retrieval and analysis.
-- 5). How do you optimize a SQL query?
-- SQL query optimization is the process of improving query performance to retrieve data faster and reduce resource consumption.

-- Important techniques include:
-- 1. Indexing: Helps retrieve records faster.
-- 2. *Avoid SELECT : Retrieve only the required columns.
-- 3. Use WHERE: Filter unnecessary records.
```

```
-- 4. Avoid unnecessary JOINs: Use only required table combinations.
-- 5. Use LIMIT: Restrict the number of returned records when appropriate.
-- 6. Use EXPLAIN: Analyze how the database executes a query.
-- Query optimization improves database performance and makes data retrieval more efficient.
-- 6). What6.  is a View in SQL?
-- A view is a virtual table created using an SQL query. It displays data from one or more existing tables.
-- A view generally does not store the actual data separately. Instead, it retrieves information from the underlying tables.

-- Advantages of views:
-- 1. Simplifies complex SQL queries.
-- 2. Improves data security.
-- 3. Provides easy access to specific information.
-- 4. Reduces query repetition.
-- 5. Makes data analysis easier.
-- Views are commonly used to simplify database operations and present selected information to users.
-- 7). How would you handle NULL values in SQL?
-- NULL represents missing, unknown, or unavailable data in a database.

-- NULL values should be handled carefully because they are different from zero or an empty string.

-- Common methods of handling NULL values include:
-- 1. IS NULL: Identifies records containing NULL values.
-- 2. IS NOT NULL: Identifies records that do not contain NULL values.
-- 3. COALESCE(): Returns the first non-NULL value from a list of expressions.
-- 4. IFNULL(): Replaces a NULL value with a specified alternative in MySQL.
-- 5. NULLIF(): Returns NULL when two expressions are equal.
```